



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Dancel, Jamella Tracy L.
Section:	AN42
Professor:	Ms. Crisola G. Tan
Date:	October 5,2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**
(Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF file**.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# ----- Get user input from console -----
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# ----- Agent Definition -----
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# ----- Model Definition -----
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                             random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# ----- Parameters from user input -----
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

# ----- Run Model -----
```

```

model = RandomWalkModel(parameters)
model.setup()

# ----- Interactive Animation -----
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200) # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()

```

2. Run the program in your terminal.
3. Enter the following values when prompted:
 - **Number of agents** (e.g., 5)
 - **Grid size** (e.g., 10 for a 10x10 grid)
 - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
 - How agents spread across the grid over time
 - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:
 - **More agents** (e.g., 10 or 20)
 - **Smaller/larger grid sizes**
 - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths:** Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules:** Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions:** Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios:** Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks) ↴

1. Install required dependency for Google Colab

```
!pip install -q agentpy
```

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from IPython.display import HTML
```

----- Get user input from console -----

```
try:
```

```
    num_agents = int(input("Enter number of agents (e.g., 5): ") or 5)
```

```
    grid_size = int(input("Enter grid size (e.g., 10 for 10x10): ") or 10)
```

```
    num_steps = int(input("Enter number of steps (e.g., 20): ") or 20)
```

```
except ValueError:
```

```
    print("Invalid input! Falling back to defaults: 5 agents, 10x10 grid, 20 steps.")
```

```
    num_agents, grid_size, num_steps = 5, 10, 20
```

----- Agent Definition -----

```
class RandomWalker(ap.Agent):
```

```
    def step(self):
```

```
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
```

```
        x, y = self.position
```

```
        # Keep agents inside grid
```

```
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
```

```
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
```

```
        self.position = (x, y)
```

----- Model Definition -----

```
class RandomWalkModel(ap.Model):
```

```

def setup(self):
    # Create agents
    self.agents = ap.AgentList(self, self.p.agents, RandomWalker)
    # Initialize random positions
    for agent in self.agents:
        agent.position = (random.randint(0, self.p.grid_size[0]-1),
                        random.randint(0, self.p.grid_size[1]-1))
    # Create grid
    self.grid = ap.Grid(self, self.p.grid_size, torus=False)
    self.grid.add_agents(self.agents)

def step(self):
    for agent in self.agents:
        agent.step()

# ----- Parameters from user input -----
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
}

# ----- Run Model -----
model = RandomWalkModel(parameters)
model.setup()

# ----- Interactive Animation (Colab/iPad Compatible) -----
fig, ax = plt.subplots(figsize=(6, 6))
ax.set_xlim(-0.5, model.p.grid_size[0] - 0.5)
ax.set_ylim(-0.5, model.p.grid_size[1] - 0.5)
ax.set_xticks(range(model.p.grid_size[0]))
ax.set_yticks(range(model.p.grid_size[1]))
ax.grid(True)
ax.set_title("Random Walk Simulation")

scat = ax.scatter([], [], s=200, color='crimson')

def update(frame):
    if frame > 0:
        model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]

```

```

    scat.set_offsets(list(zip(x, y)))
    return scat,

plt.close(fig)

# Generate HTML5 video player compatible with iPad Safari/Colab
ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True,
repeat=False)
HTML(ani.to_jshtml())

```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

Answer the Questions ↓

1. When you pack a huge number of agents into a small grid, things get crowded fast. In a simple setup where agents can overlap, you just end up with a bunch of dots stacking on top of each other in the same spots. But if you add a rule where agents can't share a space, high density creates immediate spatial gridlock. Adjacent spaces fill up quickly, causing agents to constantly bump into one another, slow down, and get stuck in virtual traffic jams.
2. The number of steps in the simulation acts as a timeline for how far agents can travel from where they started. In the first few steps, all the agents stay clumped near their initial spawn points. As time goes on and they keep taking random steps, their paths branch out, and they gradually disperse across the board. Given enough steps, the agents eventually cover the entire grid, spreading out until their overall distribution looks mostly uniform across the available space.
3. Despite all that random bouncing around, the agents never actually escape the borders of the grid. This happens because the simulation enforces strict boundary limits behind the scenes. Whenever an agent tries to take a step that would push it beyond the edge, the system catches that coordinate and caps it right at the outer boundary line. Essentially, the edges of the grid act like invisible physical walls, keeping every agent safely contained inside the designated area no matter how far or fast they try to move.
4. Even though this model is simple, its underlying concept is super useful for representing real-world systems driven by random movement. In physics and chemistry, it helps explain diffusion and Brownian motion, like how a drop of ink gradually spreads out in a glass of water. Urban planners and engineers use similar agent-based ideas to study crowd dynamics, observing how groups of people navigate through tight corridors or clear out during emergency evacuations. Beyond that, the same basic principles can be

tweaked to model how animals search for food in the wild or how infectious diseases spread across a moving population.